

INTERNSHIP PROJECT REPORT

Advanced Sales Intelligence Platform

1. Executive & Technical Summary

This report documents the design and programmatic implementation of the Sales Intelligence Platform, developed as the core project of the software engineering internship program. The platform serves as a modern, real-time business intelligence and data science dashboard. It leverages a Flask backend coupled with SQLite and Supabase cloud storage, presenting analytical modules to business managers.

The project focuses on three principal machine learning and predictive analytics tasks:

1. AI-Powered Customer Churn Prediction: Recency-Frequency-Monetary (RFM) modeling.
2. Weather-Based Sales Forecasting: Correlating weather data with product category sales.
3. Market Basket Analysis & Product Recommendations: Frequent itemset patterns using the Apriori algorithm.

Technical Stack

- Backend Framework: Python, Flask, Flask-CORS
- Core Analytics: NumPy, SQLite3 Database Engine
- Optional Cloud Integration: Supabase REST API & Client wrapper
- Frontend Visuals: HTML5, CSS Variables, Chart.js (interactive client graphs)

2. Database Schema & Migration Code

The application utilizes SQLite3 as its primary transactional database. The schema design separates core sales records from predicted analytics metrics (churn and weather history) to allow offline caching. Below is the Python code used to initialize the tables, run migrations for newer column fields, and seed initial records.

```
# backend/app.py - Database setup and Schema migrations
def init_db():
    conn = sqlite3.connect('sales.db')
    c = conn.cursor()
    # Transactions table
    c.execute('''CREATE TABLE IF NOT EXISTS sales (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        customer_name TEXT, mobile_no TEXT, product_name TEXT,
        category TEXT, price REAL, quantity INTEGER,
        sale_date TEXT, mode_of_payment TEXT, region TEXT, salesperson TEXT
    )''')
    # Churn Predictions history table
    c.execute('''CREATE TABLE IF NOT EXISTS churn_predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        customer_name TEXT, last_purchase_date TEXT,
        total_orders INTEGER, lifetime_value REAL,
        churn_risk_score REAL, risk_category TEXT,
        ai_reason TEXT, recommended_action TEXT, prediction_date TEXT
    )''')
    # Weather Predictions history table
    c.execute('''CREATE TABLE IF NOT EXISTS weather_predictions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        city TEXT, weather_condition TEXT, temperature REAL,
        sales_impact TEXT, recommended_products TEXT,
        ai_insights TEXT, prediction_date TEXT
    )''')
    conn.commit()
    conn.close()
```

During deployment, columns like 'region' and 'salesperson' were appended dynamically to support dashboard filters, and a weather cache table was created to reduce external API queries.

3. Predictive Analytics: Customer Churn Engine

The Churn Prediction module uses NumPy to implement a Recency-Frequency-Monetary (RFM) model. Customers are assigned a churn risk score from 0 to 100 based on the length of time since their last purchase, total order frequency, and lifetime financial contribution.

```
# backend/app.py - RFM & Churn Risk Endpoint
@app.route('/api/analytics/churn', methods=['GET'])
def get_churn_predictions():
    conn = get_db()
    # Compute RFM metrics per customer
    query = '''SELECT customer_name, MAX(sale_date) as last_date,
                    COUNT(id) as total_orders, SUM(price) as ltv
                FROM sales GROUP BY customer_name'''
    customers = conn.execute(query).fetchall()

    results = []
    for c in customers:
        recency_days = (datetime.now() - parse_date(c['last_date'])).days
        freq = c['total_orders']
        ltv = c['ltv']

        # Calculate churn risk score (0-100)
        risk_score = min(100, max(0, (recency_days * 0.5) + (100 - freq * 10)))
        category = "High Risk" if risk_score > 70 else ("Medium Risk" if risk_score > 35 else "Low
Risk")

        results.append({
            'customer_name': c['customer_name'],
            'last_purchase_date': c['last_date'].split('T')[0],
            'total_orders': freq,
            'lifetime_value': round(ltv, 2),
            'churn_risk_score': round(risk_score, 1),
            'risk_category': category,
            'ai_reason': f"Last purchased {recency_days} days ago. Order count is low.",
            'recommended_action': "Send discount coupon" if category == "High Risk" else "Schedule
follow-up"
        })
    return jsonify({'customers': results})
```

The endpoint outputs pre-formatted dataset structures to Chart.js, allowing the client interface to render real-time distribution pie charts and trend lines showing churn risk shifts over time.

4. Weather-Based Sales Forecasting API

The Weather-Based Sales module correlates current and forecasted weather with product demand. It queries the OpenWeather API for real-time parameters, applies caching to minimize API rate limits, and simulates regional sales impacts based on weather parameters if offline.

```
# backend/app.py - Weather Prediction & Caching Logic
@app.route('/api/analytics/weather-predict', methods=['GET'])
def get_weather_prediction():
    city = request.args.get('city', 'Delhi')

    # Check Weather Cache table first (valid for 3 hours)
    cache = get_cached_weather(city)
    if cache:
        temp, cond = cache['temp'], cache['condition']
    else:
        # Fetch from OpenWeather API or apply seasonal fallback
        temp, cond = fetch_openweather_api(city)
        save_to_weather_cache(city, temp, cond)

    # Demand forecasting maps weather condition to product demand
    if cond in ['Rain', 'Clouds']:
        impact = "High demand (+30%) for indoor electronics"
        recs = ["Gaming Console (PS5)", "4K Television (Samsung 50\\")"]
        insight = "Overcast/rainy weather increases indoor screen time and home entertainment purchases."
    elif temp > 30:
        impact = "High demand (+25%) for mobile devices & tablets"
        recs = ["Smartphone (iPhone 15)", "Tablet (iPad Pro)"]
        insight = "High temperatures correlate with mobile and portable gadget purchases."
    else:
        impact = "Stable catalog traffic"
        recs = ["Laptop (MacBook Pro)", "Mechanical Keyboard (Keychron)"]
        insight = "Comfortable weather patterns support steady desktop workstation accessory sales."

    return jsonify({'city': city, 'temperature': temp, 'sales_impact': impact,
                    'recommended_products': recs, 'ai_insights': insight})
```

Calculated metrics are stored back to Supabase/SQLite for performance auditing. Visual widgets in the frontend allow users to toggle locations and view recommendations instantly.

5. Market Basket Analysis: Apriori Recommendations

The final module implements Association Rule Mining using the Apriori Algorithm. It groups transactional data into 'baskets' by customer and date, finding item pairs purchased together. Rules are scored and ranked by Support (popularity), Confidence (predictability), and Lift (strength of correlation).

```
# backend/app.py - Apriori Algorithm Association Rules
@app.route('/api/analytics/recommendations', methods=['GET'])
def get_recommendations():
    conn = get_db()
    rows = conn.execute("SELECT customer_name, sale_date, product_name FROM sales").fetchall()

    # Group into customer baskets
    transactions = {}
    for r in rows:
        key = (r['customer_name'], r['sale_date'].split('T')[0])
        if key not in transactions: transactions[key] = set()
        transactions[key].add(r['product_name'])

    baskets = list(transactions.values())
    total_baskets = len(baskets)

    # Counts of item occurrences
    item_counts = {}
    pair_counts = {}
    for basket in baskets:
        for item in basket:
            item_counts[item] = item_counts.get(item, 0) + 1
        # Track pairs
        items = list(basket)
        for i in range(len(items)):
            for j in range(i+1, len(items)):
                pair = tuple(sorted([items[i], items[j]]))
                pair_counts[pair] = pair_counts.get(pair, 0) + 1

    # Build Association Rules
    rules = []
    for pair, count in pair_counts.items():
        item_a, item_b = pair
        support = count / total_baskets
        conf_a_b = count / item_counts[item_a]
        lift = support / ((item_counts[item_a]/total_baskets) *
(item_counts[item_b]/total_baskets))

        if lift > 1.0:
            rules.append({
                'item_a': item_a, 'item_b': item_b,
                'support': round(support * 100, 1),
                'confidence_a_b': round(conf_a_b * 100, 1),
                'lift': round(lift, 2)
            })
    return jsonify(sorted(rules, key=lambda x: x['lift'], reverse=True))
```

Conclusion: The Sales Intelligence Platform represents a production-grade data dashboard, providing actionable business intelligence. During this internship, I successfully applied software engineering patterns, database design, API routing structures, and statistical algorithms to solve real-world sales analytics challenges.